

Chain of Thought (CoT)

Scenario:

Customer tries to transfer ₹50,000.

Transaction failed.

Error shown: "Transaction could not be completed."

Expected discoveries:

1. Daily limit exceeded
2. Insufficient balance
3. Bank server unavailable
4. OTP expired
5. Network timeout
6. Beneficiary inactive
7. Fraud rule triggered
8. Core banking unavailable
9. UPI switch unavailable

Answer from ChatGPT:

This scenario is **ambiguous** because the UI shows only a generic error. The automation/test design should verify which underlying condition caused the failure.

Possible failure reasons

# Failure condition	Expected behavior
1 Daily transfer limit exceeded	Transaction rejected with limit-exceeded message
2 Insufficient balance	Transaction rejected due to insufficient funds
3 Bank server unavailable	Transaction fails gracefully
4 OTP expired	Transaction rejected and user asked to generate a new OTP
5 Network timeout	Transaction fails/timeout is handled gracefully
6 Beneficiary inactive	Transfer rejected because beneficiary is inactive
7 Fraud rule triggered	Transaction blocked and appropriate security message shown

# Failure condition	Expected behavior
8 Core banking unavailable	Transaction fails gracefully
9 UPI switch unavailable	Transaction fails gracefully

Important test-design point

The message "**Transaction could not be completed.**" should **not** be treated as sufficient validation for all nine cases.

For each scenario, the test should validate:

1. Transfer amount = **₹50,000**
2. Transfer attempt is initiated.
3. Transaction fails.
4. Generic error message is displayed.
5. The **actual underlying failure reason** is verified through an appropriate test hook, API response, database record, mock/stub, or backend test data.
6. No duplicate transaction is created.
7. Customer balance is not incorrectly debited.
8. Transaction status is recorded correctly as **FAILED**.

If this is for **test-case design**, I can next convert these 9 failure reasons into a complete **positive/negative test-case matrix with Test Case ID, preconditions, test data, steps, expected result, and priority**.

Absolutely. Below is a **complete negative test-case matrix** for the ₹50,000 transfer scenario.

Transaction Failure Test Cases

TC ID	Failure Scenario	Preconditions / Test Data	Test Steps	Expected Result	Priority
TC_001	Daily limit exceeded	Daily limit = ₹40,000; transfer = ₹50,000	Login → Select beneficiary → Enter ₹50,000 → Initiate transfer	Transaction fails. User sees transaction failure message. Backend records DAILY_LIMIT_EXCEEDED . No amount is debited.	P0
TC_002	Insufficient balance	Available balance = ₹30,000; transfer = ₹50,000	Login → Enter ₹50,000 → Initiate transfer	Transaction fails due to insufficient balance. No amount is debited.	P0

TC ID	Failure Scenario	Preconditions / Test Data	Test Steps	Expected Result	Priority
TC_003	Bank server unavailable	Bank service returns 5xx/service unavailable	Initiate ₹50,000 transfer	Transaction fails gracefully. Appropriate error is displayed. No partial debit occurs.	P0
TC_004	OTP expired	Valid transfer request; OTP generated but expired	Enter ₹50,000 → Request OTP → Wait/force OTP expiry → Enter expired OTP	Transaction is rejected. User is informed that OTP has expired and can request a new OTP. No debit occurs.	P1
TC_005	Network timeout	Network timeout configured during transaction	Enter ₹50,000 → Initiate transfer → Simulate network timeout	Timeout/failure is handled gracefully. No duplicate or partial transaction occurs. Final transaction status is correctly recorded.	P0
TC_006	Beneficiary inactive	Beneficiary status = INACTIVE	Select inactive beneficiary → Enter ₹50,000 → Initiate transfer	Transaction is rejected because beneficiary is inactive. No debit occurs.	P0
TC_007	Fraud rule triggered	Configure transaction to trigger fraud rule	Enter ₹50,000 → Initiate transfer	Transaction is blocked. Appropriate security/fraud response is generated. No unauthorized debit occurs.	P0
TC_008	Core banking unavailable	Core banking service unavailable	Enter ₹50,000 → Initiate transfer	Transaction fails gracefully. Correct failure status is recorded. No incorrect debit occurs.	P0
TC_009	UPI switch unavailable	UPI switch/service unavailable	Enter ₹50,000 → Initiate transfer	Transaction fails gracefully with appropriate error handling. No duplicate/partial debit occurs.	P0

Common validation for all 9 cases

For every test case, verify these common conditions:

- **Transfer amount:** ₹50,000
- Transaction should **not be completed**
- Failure should be recorded with the correct **backend reason/code**
- Customer account should **not be incorrectly debited**
- Beneficiary should not receive the amount
- No duplicate transaction should be created

- Transaction ID/reference should be handled correctly
- Transaction history should show the appropriate **FAILED/REJECTED** status
- Generic UI message "Transaction could not be completed." may be displayed, but the **underlying failure reason must be independently validated**
- Appropriate logs/audit records should be generated

Test data matrix

Scenario	Account Balance	Daily Limit	Beneficiary	External Service	Expected Failure Code
Daily limit	₹1,00,000	₹40,000	Active	Available	DAILY_LIMIT_EXCEEDED
Insufficient balance	₹30,000	₹1,00,000	Active	Available	INSUFFICIENT_BALANCE
Bank server	₹1,00,000	₹1,00,000	Active	Bank DOWN	BANK_SERVER_UNAVAILABLE
OTP expired	₹1,00,000	₹1,00,000	Active	Available	OTP_EXPIRED
Network timeout	₹1,00,000	₹1,00,000	Active	Timeout	NETWORK_TIMEOUT
Beneficiary inactive	₹1,00,000	₹1,00,000	Inactive	Available	BENEFICIARY_INACTIVE
Fraud rule	₹1,00,000	₹1,00,000	Active	Fraud rule triggered	FRAUD_RULE_TRIGGERED
Core banking	₹1,00,000	₹1,00,000	Active	Core banking DOWN	CORE_BANKING_UNAVAILABLE
UPI switch	₹1,00,000	₹1,00,000	Active	UPI switch DOWN	UPI_SWITCH_UNAVAILABLE

Key automation principle: Don't create nine UI tests that all assert only "Transaction could not be completed." The UI behavior is largely identical; the **test data/backend condition** should drive the different failure paths, while API/database/service-level validation confirms the actual reason.